Video Framework Comparison

iOS Core Video vs Android Multimedia

Overview | Architecture | Components | Pipeline | Performance | Code Examples

Core Video

Apple's framework for working with digital video. Provides a pipeline model for processing video frames with optimal performance.

Android Multimedia

Android's comprehensive multimedia framework including MediaCodec, MediaPlayer, ExoPlayer, and Camera2 API for video processing and playback.

Core Video

Apple's framework for working with digital video. Provides a pipeline model for processing video frames with optimal performance.

Hardware Accelerated
Deep integration with Metal and GPU

Zero-Copy Buffers
Efficient memory management with CVPixelBuffer

Display Sync
CVDisplayLink for smooth rendering

Android Multimedia

Android's comprehensive multimedia framework including MediaCodec, MediaPlayer, ExoPlayer, and Camera2 API for video processing and playback.

Hardware Codecs
MediaCodec API with hardware acceleration

Surface Rendering
Direct rendering to Android Surface

Flexible Pipeline
Multiple APIs for different use cases

Video Framework Comparison

iOS Core Video vs Android Multimedia

Overview | **Architecture** | Components | Pipeline | Performance | Code Examples

iOS Core Video Architecture

Core Foundation Layer
CVPixelBuffer, CVImageBuffer

Processing Layer
Core Image, vImage, Accelerate

Graphics Integration
Metal, OpenGL ES textures

Display Layer
CVDisplayLink, CADisplayLink

Capture Layer
AVCaptureVideoDataOutput

Android Multimedia Architecture

Application Layer
MediaPlayer, ExoPlayer, CameraX

Framework Layer
MediaCodec, AudioTrack

Native Layer
OpenMAX, NDK APIs

HAL Layer
Hardware Abstraction Layer

Hardware Layer
GPU, DSP, Dedicated codecs

Video Framework Comparison

iOS Core Video vs Android Multimedia

Overview | Architecture | **Components** | Pipeline | Performance | Code Examples

Key Components Comparison

Buffer Management

iOS	Android	Common Features
CVPixelBuffer, CVPixelBufferPool	ByteBuffer, MediaCodec buffers	Efficient memory pools, hardware-backed buffers

Video Decoding

iOS	Android	Common Features
AVAssetReader, VTDecompressionSession	MediaCodec, MediaExtractor	Hardware acceleration, format negotiation

Video Encoding

iOS	Android	Common Features
AVAssetWriter, VTCompressionSession	MediaCodec, MediaMuxer	Bitrate control, quality settings

Camera Access

iOS	Android	Common Features
AVCaptureSession, AVCaptureDevice	Camera2, CameraX	Manual controls, multiple streams

Graphics Integration

iOS	Android	Common Features
CVMetalTextureCache	SurfaceTexture, ImageReader	GPU texture sharing, zero-copy rendering

Video Framework Comparison

iOS Core Video vs Android Multimedia

Overview | Architecture | Components | **Pipeline** | Performance | Code Examples

iOS Video Pipeline

Camera/File → AVCapture → CVPixelBuffer → Core Image → Metal/GL → Display

Android Video Pipeline

Camera/File → Camera2/MediaExtractor → MediaCodec → ImageReader → OpenGL/Vulkan → Surface

Video Framework Comparison

iOS Core Video vs Android Multimedia

Overview | Architecture | Components | Pipeline | **Performance** | Code Examples

Hardware Acceleration

iOS: Metal integration	95%
Android: Hardware codecs	90%

Memory Efficiency

iOS: CVPixelBufferPool	92%
Android: Buffer sharing	88%

Latency (Lower is Better)

iOS: Real-time processing	85%
Android: Camera2 API	82%

Format Support

iOS: H.264, HEVC, ProRes	88%
Android: Varies by device	85%

Video Framework Comparison

iOS Core Video vs Android Multimedia

Overview | Architecture | Components | Pipeline | Performance | **Code Examples**

iOS: Creating CVPixelBuffer

```
// Create pixel buffer
var pixelBuffer: CVPixelBuffer?
let attrs = [
  kCVPixelBufferPixelFormatTypeKey:
    kCVPixelFormatType_32BGRA,
  kCVPixelBufferMetalCompatibilityKey: true
] as CFDictionary

CVPixelBufferCreate(
  kCFAllocatorDefault,
  width, height,
  kCVPixelFormatType_32BGRA,
  attrs,
  &pixelBuffer
)

// Convert to Metal texture
let textureCache =
  CVMetalTextureCacheCreate(...)
CVMetalTextureCacheCreateTexture(
  from: pixelBuffer
)
```

Android: MediaCodec Setup

```
// Create decoder
MediaCodec decoder =
  MediaCodec.createDecoderByType(
    "video/avc"
  );

MediaFormat format =
  MediaFormat.createVideoFormat(
    "video/avc", width, height
  );

decoder.configure(
  format, surface, null, 0
);
decoder.start();

// Process buffers
int inputIndex =
  decoder.dequeueInputBuffer(timeout);
ByteBuffer buffer =
  decoder.getInputBuffer(inputIndex);
decoder.queueInputBuffer(...);
```

iOS: Real-time Processing

```
let output =
  AVCaptureVideoDataOutput()
output.setSampleBufferDelegate(
  self,
  queue: videoQueue
)

func captureOutput(
  _ output: AVCaptureOutput,
  didOutput sampleBuffer:
    CMSampleBuffer
) {
  guard let pixelBuffer =
    CMSampleBufferGetImageBuffer(
      sampleBuffer
    ) else { return }

  // Process with Core Image
  let ciImage =
    CIImage(cvPixelBuffer: pixelBuffer)
  let filtered = filter.outputImage
}
```

Android: Camera2 Processing

```
ImageReader reader =
  ImageReader.newInstance(
    width, height,
    ImageFormat.YUV_420_888, 2
  );

reader.setOnImageAvailableListener(
  imageReader -> {
    Image image =
      imageReader.acquireLatestImage();

    // Process YUV planes
    Image.Plane[] planes =
      image.getPlanes();
    ByteBuffer yBuffer =
      planes[0].getBuffer();

    // Apply processing
    processFrame(yBuffer);
    image.close();
  }, handler
);
```